Addis Ababa University • School of Information Technology and Engineering

Selenium End-to-End Automated Testing Report

Student Name & ID: Peter Kinfe (ATE/7749/15)	**Target Website:** ParaBank (https://parabank.parasoft.com/parabank)
Course: Software Testing and Validation	**Tech Stack:** Java 21, Selenium WebDriver 4.28, JUnit 5.11, Maven 3.9
GitHub Repo: https://github.com/p3ter-dev/selenium_ate_7749_15	**Suite Result:** 11/11 Tests Passed

1. Website Selection and Justification

ParaBank by Parasoft is an interactive public banking demonstration platform specifically architected for automated testing. It was selected because it delivers rich state transitions across account registration, multi-session authentication, asynchronous AJAX fund transfers, and form validation error messaging without CAPTCHA or anti-bot disruptions. This allows robust end-to-end verification of complex web interactions.

2. Test Case Execution Summary (T1 – T8)

ID	Action / Flow	Input Data	Expected Result	Actual Result	Status
T1	Navigation Smoke Test	GET /index.htm	Title equals 'ParaBank | Welcome | Online Banking'; logo displayed	Page loaded; title and header match exactly	**PASS**
T2	Locator Strategies	By.name, By.cssSelector, By.className, By.id, By.xpath	All elements resolved uniquely without positional XPaths	Resolved 5 distinct elements across multiple locator types	**PASS**
T3	Main Positive E2E Path	Dynamic user registration + $150.00 transfer	Account created; transfer executed; 'Transfer Complete!' header	Registration and fund transfer completed with verified confirmation	**PASS**
T4	Negative Path (Auth & Forms)	Invalid login + empty registration form	Displays 'The username and password could not be verified' & field errors	Authentication error and required field validations displayed correctly	**PASS**
T5	Explicit Wait Synchronization	WebDriverWait with ExpectedConditions ($75.50 transfer)	Dynamic AJAX response element synchronized without Thread.sleep	Synchronized explicitly with dropdown load and result banner	**PASS**
T6	Data-Driven EP Parameterized	5 equivalence classes (valid, non-existent, empty inputs)	Correct state branching (successful login vs targeted error message)	All 5 parameterized partitions executed and asserted expected state	**PASS**
T7	Page Object Model (POM)	LoginPage, RegisterPage, OverviewPage, TransferPage	Tests call intention-revealing methods without leaking raw locators	Clean encapsulation of actions, elements, and assertions	**PASS**
T8	JUnit Lifecycle & Execution	@BeforeEach, @AfterEach, mvn test CLI	Fresh browser per test, clean teardown, runs in headless/UI modes	Suite passes with 0 failures via standard Maven lifecycle	**PASS**

3. Test Design Technique for T6: Equivalence Partitioning (EP)

Equivalence Partitioning (EP) was applied to the Authentication and Login subsystem. The input domain of credential combinations was divided into distinct equivalence classes where the system is expected to behave uniformly. Testing a representative value from each partition provides comprehensive defect detection with optimal test efficiency.

Partition ID	Equivalence Class	Representative Input	Expected Output / System Behavior	Result
EP1 (Valid)	Valid Registered User + Valid Password	user: *dynamic_user*, pass: *Password123!*	Successful authentication; redirects to Accounts Overview dashboard	**PASS**
EP2 (Invalid)	Non-existent Username + Arbitrary Password	user: *invalid_user_99*, pass: *wrongpass*	Error: 'The username and password could not be verified.'	**PASS**
EP3 (Invalid)	Empty Username + Non-empty Password	user: *""*, pass: *SomePassword123!*	Error: 'Please enter a username and password.'	**PASS**
EP4 (Invalid)	Non-empty Username + Empty Password	user: *some_user*, pass: *""*	Error: 'Please enter a username and password.'	**PASS**
EP5 (Invalid)	Empty Username + Empty Password	user: *""*, pass: *""*	Error: 'Please enter a username and password.'	**PASS**

4. Architectural Design & Page Object Model

The test suite follows the industry-standard **Page Object Model (POM)** pattern to enforce maintainability and separation of concerns:
• **BasePage**: Centralizes `WebDriver` operations and explicit wait utilities (`waitForVisibility`, `waitForClickable`, `type`, `click`).
• **LoginPage, RegisterPage, AccountsOverviewPage, TransferFundsPage**: Encapsulate UI elements and intention-revealing business actions.
• **BaseTest**: Guarantees isolated browser lifecycle per test (`@BeforeEach` fresh instance, `@AfterEach` teardown, headless support).

5. Identified Site Behaviors and Usability Observations

• **Asynchronous Account Population Delay:** On transfer.htm, account dropdown options are populated asynchronously via an AJAX call. Submitting before options render results in a blank transfer error, necessitating explicit synchronization on dropdown option presence.
• **Lenient Input Sanitization:** The transfer amount field accepts non-numeric inputs without client-side blocking, relying entirely on server-side response handling.

6. Evidence of Green test run (the console execution log)